

Shiv Nadar University Chennai

CS3807 Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNet V2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to $224 \times 224 \times 3$.

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNet V2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

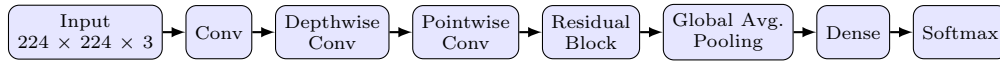
4. MobileNet V2 Architecture

MobileNet V2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNet V2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

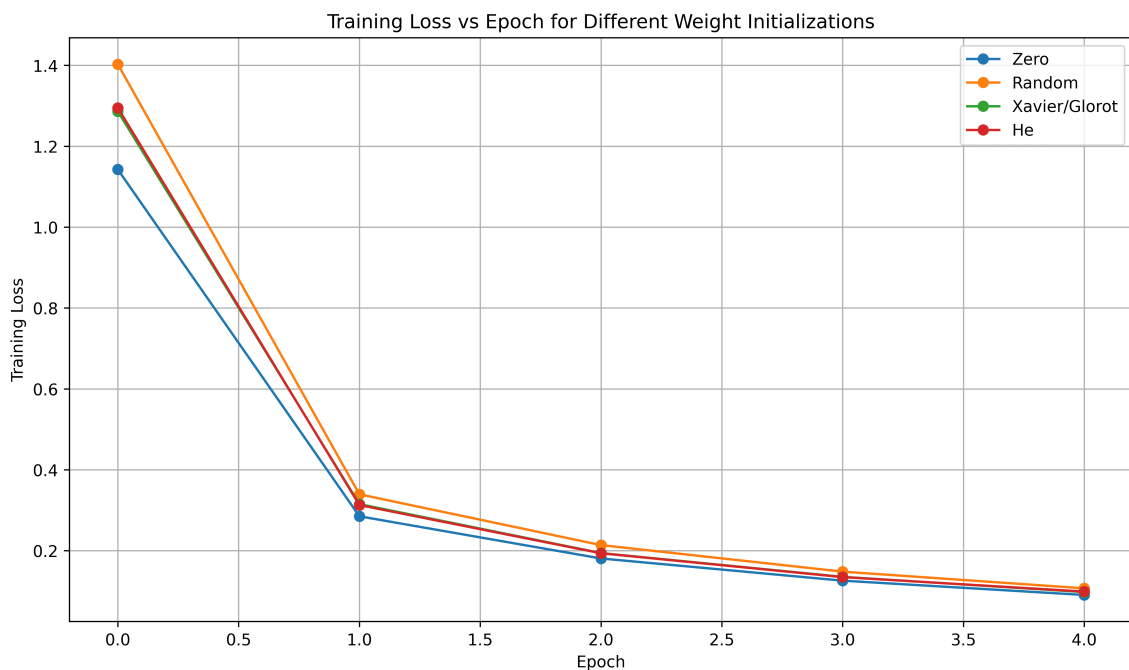


Figure 1: Training Loss vs. Epoch for Different Weight Initializations

The plot depicts the variation of the training loss of the network over the five epochs using the various weight initialization techniques. Training loss is initially high at the beginning of epoch 1, but it falls drastically and thereafter continues falling slowly. Among the weight initialization techniques used, zero initialization results in the least training loss, whereas Random initialization results in the highest training loss which is slightly higher than that of the Xavier/Glorot and He initializations. Falling of the training loss indicates learning in the network.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

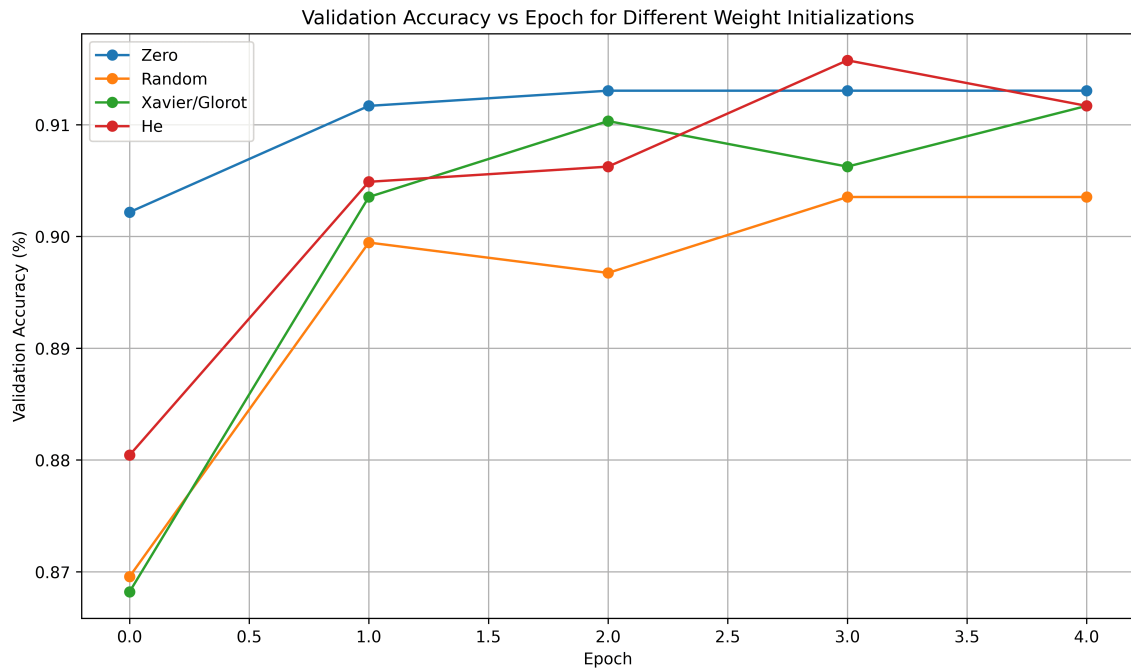


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initializations

The plot shows the validation accuracy obtained with different weight initialization methods over five epochs. Zero initialization starts with relatively high validation accuracy and remains fairly stable around 91%. He initialization improves steadily and reaches the highest validation accuracy at approximately epoch 3, while Xavier/Glorot also improves but shows a small fluctuation after epoch 2. Random initialization improves more gradually and remains lower than the other methods. This difference occurs because the initialization method determines the starting values of the weights and therefore affects how the network moves through the optimization process and learns useful features.

Students should identify which initialization gives faster and more stable convergence.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

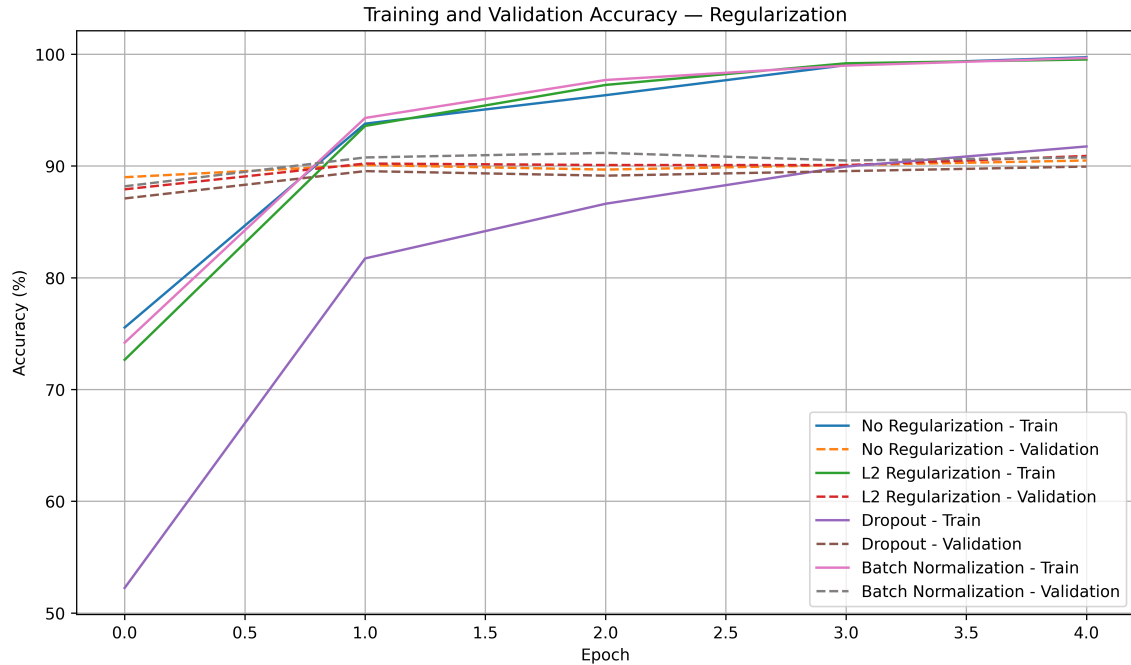


Figure 3: Training and Validation Accuracy for Different Regularization Methods

The plot compares training and validation accuracy for different regularization methods over five epochs. The training accuracy increases rapidly for all methods and approaches nearly 100%, whereas the validation accuracy remains around 89–91%. This creates a noticeable generalization gap, showing that the models fit the training data much more closely than the validation data. Dropout has a lower training accuracy because randomly deactivating neurons makes training more difficult, while Batch Normalization achieves the highest validation accuracy for much of the training. The gap between training and validation accuracy suggests some degree of overfitting, although the validation accuracy remains relatively stable within the five epochs.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

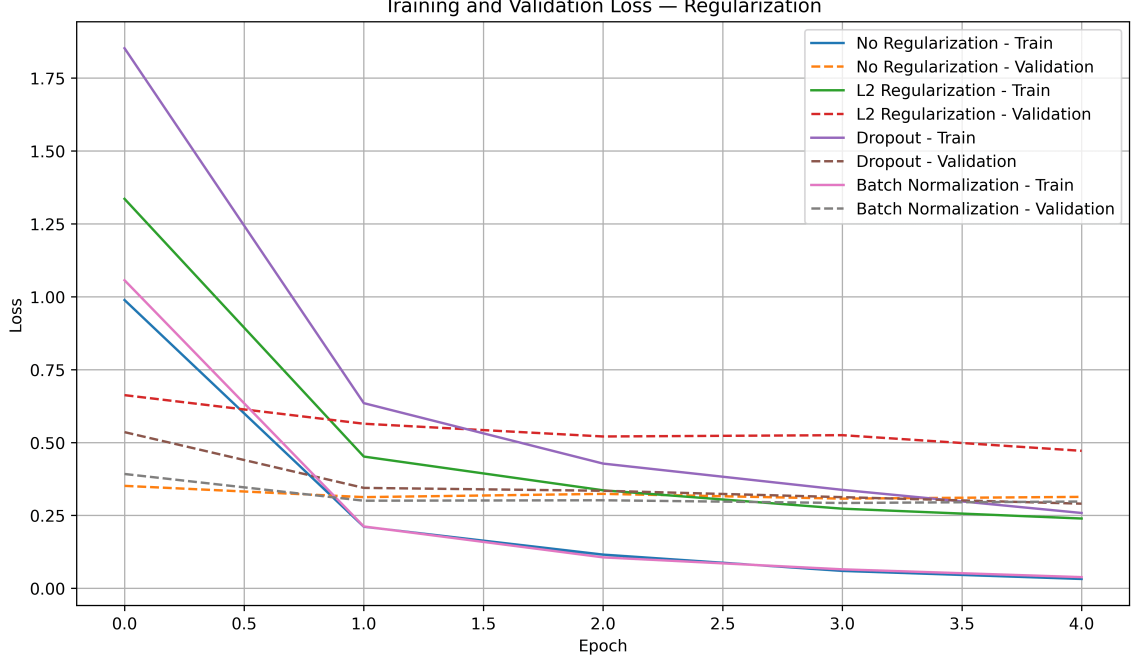


Figure 4: Training and Validation Loss for Different Regularization Methods

Plot analysis reveals that the training loss falls consistently in all the four cases, thus indicating that the models are learning from the training data. Validation losses do not increase in general but fall or remain constant as the training loss declines. The validation losses fall consistently for Dropout and Batch Normalization approaches in the late epochs, whereas the validation losses remain relatively high in case of L2 Regularization approach. The plot does not show any indication of validation loss increase for the five epochs, which means that it does not reveal anything about the problem of overfitting during this particular training period.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch $x_1, x_2, \dots, x_m$, the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider $x = [2, 4, 6, 8]$.

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore, $\sqrt{5} \approx 2.236$.

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence, $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$.

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

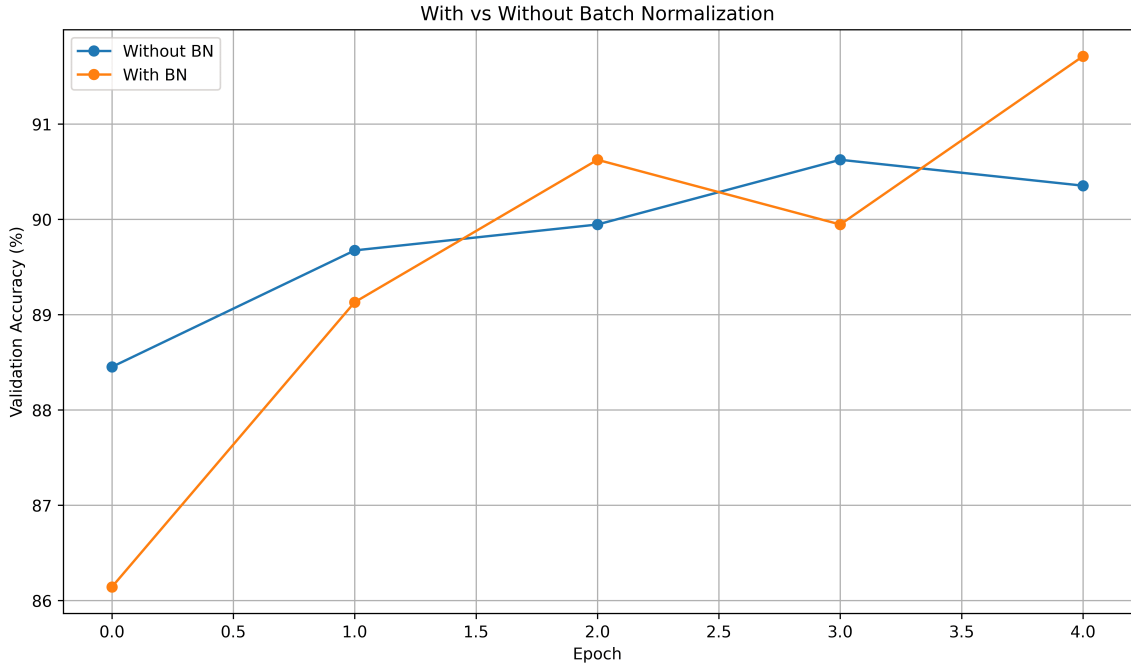


Figure 5: Validation Accuracy With and Without Batch Normalization

The plot compares the validation accuracy of the model with and without Batch Normalization across five epochs. Without Batch Normalization, the validation accuracy increases gradually from about 88.5% to 90.4%, with a small decrease after the third epoch. With Batch Normalization, the accuracy starts lower at about 86.1%, but increases more rapidly and reaches approximately 91.7% by the final epoch. The curves cross during training, showing that Batch Normalization does not necessarily give the best validation accuracy at the beginning, but can lead to better performance as training progresses. This occurs because Batch Normalization normalizes intermediate activations during training, which can make optimization more stable and allow the network to learn effectively over successive epochs. In this experiment, the higher final validation accuracy with Batch Normalization suggests better generalization than the model without it.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

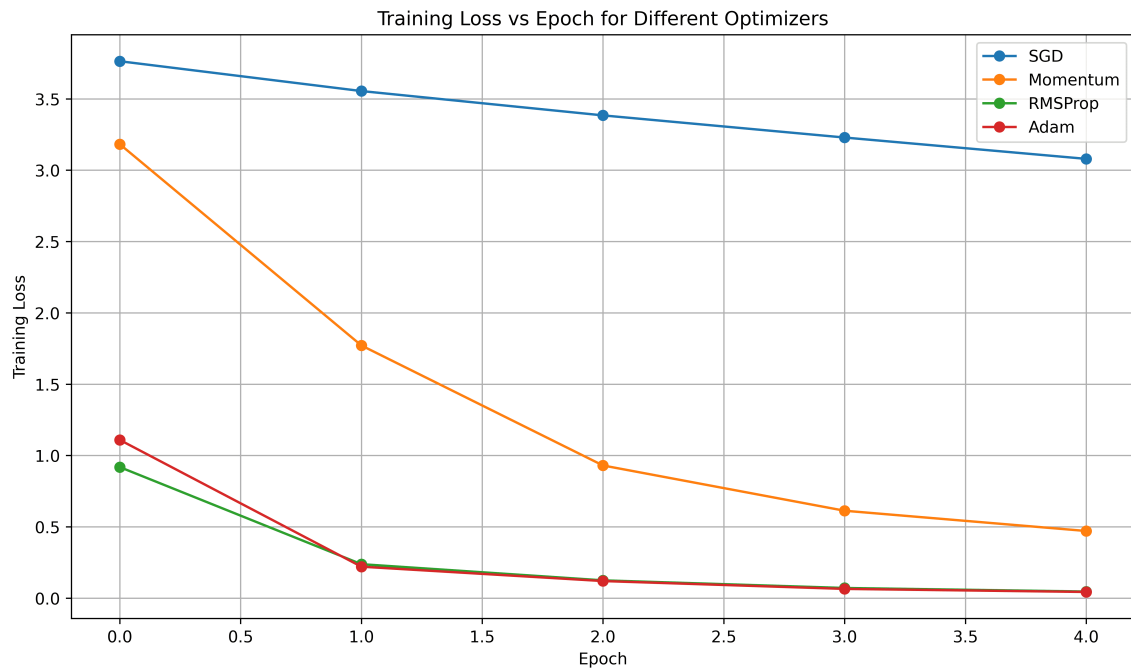


Figure 6: Training Loss vs. Epoch for Different Optimizers

The plot compares how the training loss changes over five epochs for SGD, Momentum, RMSProp and Adam. All optimizers reduce the training loss as the number of epochs increases, but their rates of convergence are very different. SGD decreases the loss only gradually, while Momentum reduces it more substantially. RMSProp and Adam show the fastest reduction, reaching a very small loss by the final epoch. This occurs because adaptive optimizers such as RMSProp and Adam adjust the effective parameter updates according to the gradients, allowing the model to make more suitable updates when different parameters have different gradient magnitudes. Momentum also accelerates learning by using information from previous updates. Therefore, the plot indicates that Adam and RMSProp converge much faster than basic SGD for this experiment.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

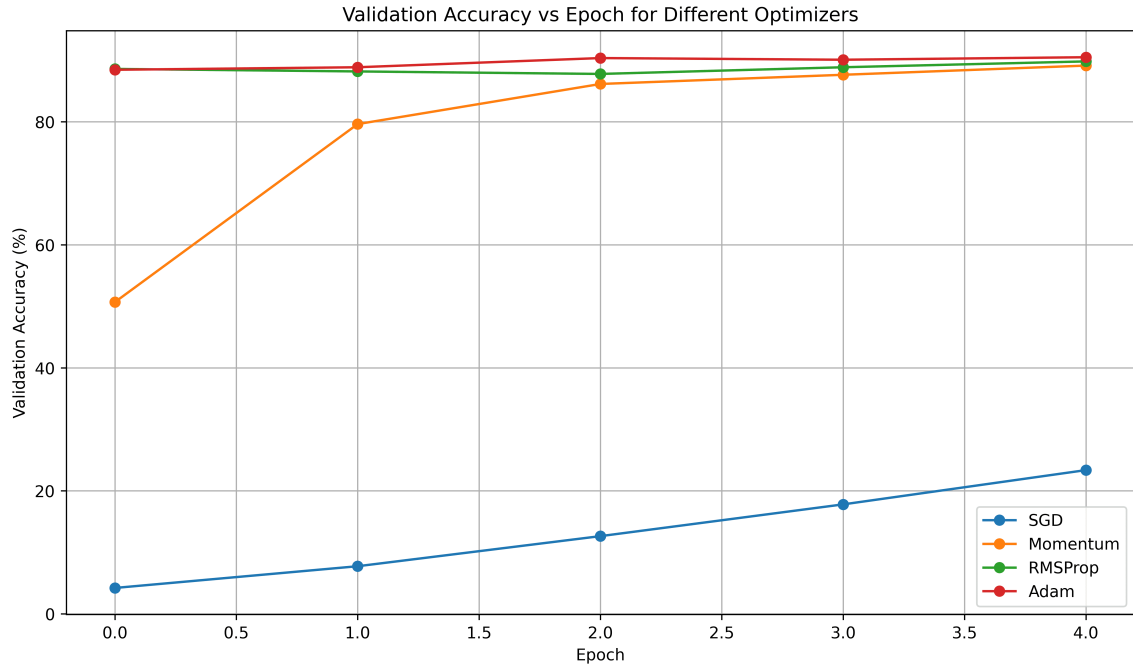


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

The plot shows the effect of the optimizer on validation accuracy during training. SGD starts with very low validation accuracy and improves slowly, reaching only about 23% after five epochs. Momentum improves rapidly from about 51% to approximately 89%. RMSProp and Adam already achieve about 88% validation accuracy at the beginning and remain close to 89% throughout training, with Adam giving the highest final accuracy. The difference occurs because the optimizers produce parameter updates with different magnitudes and directions. SGD makes relatively simple gradient-based updates, whereas Momentum, RMSProp and Adam use additional information from previous gradients or gradient magnitudes to improve the optimization process. Thus, for the fixed five-epoch training period used here, Adam provides the best combination of rapid convergence and final validation performance.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	3.0794	23.37%	3	42.87 s
Momentum	0.4708	89.13%	4	42.32 s
RMSProp	0.0464	89.81%	4	42.35 s
Adam	0.0435	90.49%	4	42.81 s

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;

- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

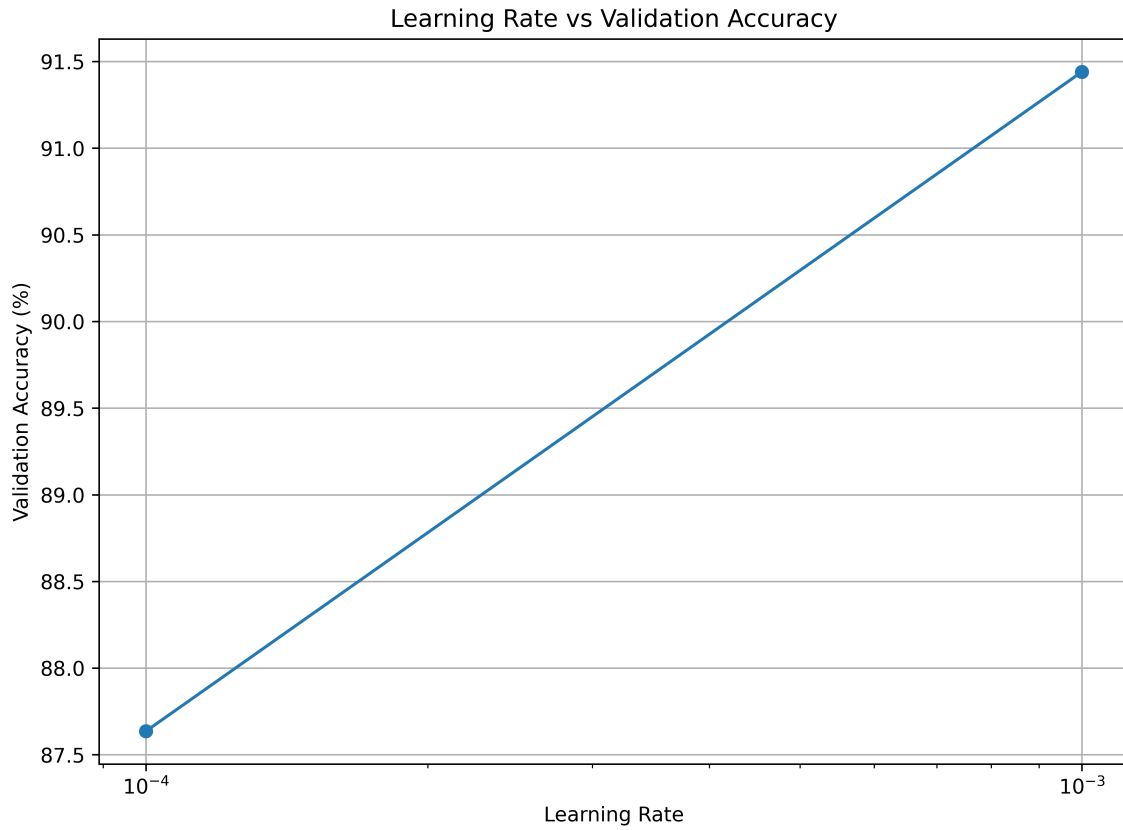


Figure 8: Learning Rate vs. Validation Accuracy

The plot shows the relationship between the learning rate and validation accuracy for the tested learning rates. Increasing the learning rate from 10^{-4} to 10^{-3} improves the validation accuracy from approximately 87.6% to 91.4%. Thus, the higher learning rate performs better for the configurations tested in this experiment. A very small learning rate produces smaller parameter updates, so the model may not learn sufficiently within the fixed number of training epochs. The result suggests that 10^{-3} provides more effective updates for this model and training setup, although the plot alone does not establish that larger learning rates would continue to improve performance.

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

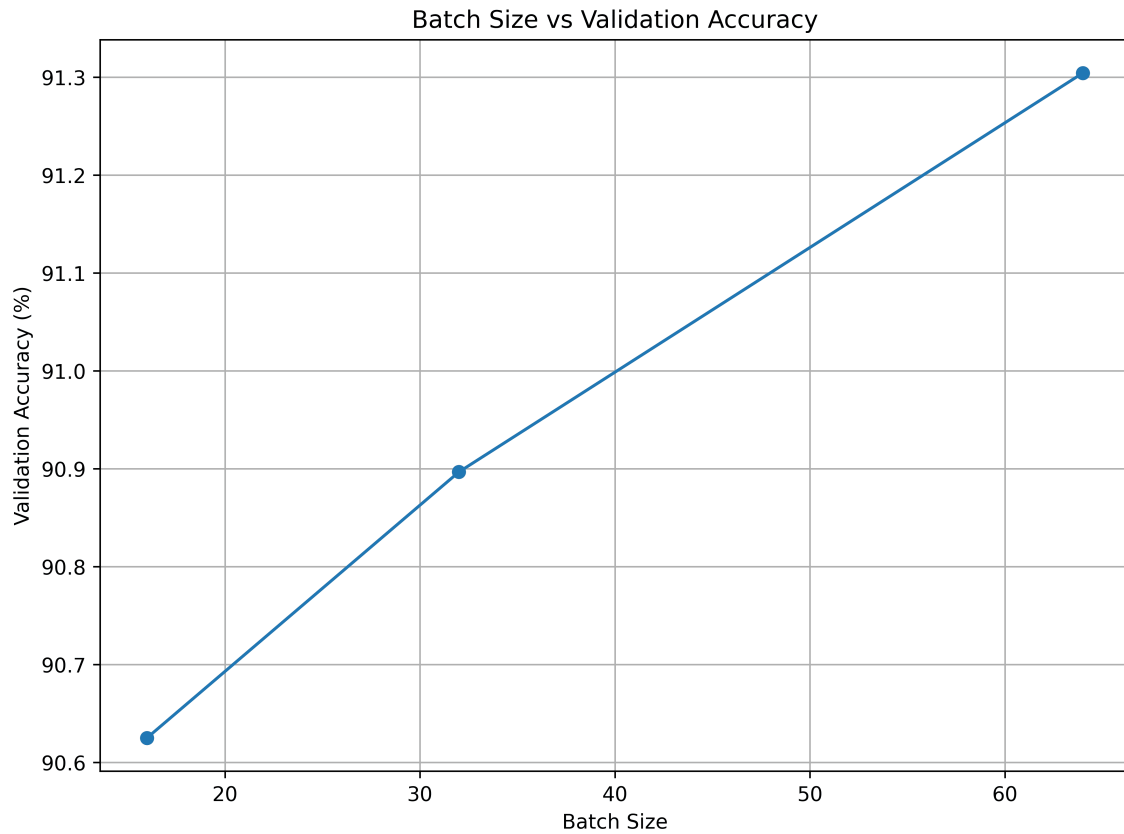


Figure 9: Batch Size vs. Validation Accuracy

The plot shows that validation accuracy increases as the batch size is increased over the tested values. The accuracy rises from approximately 90.6% for a batch size of 16 to 90.9% for 32 and then to about 91.3% for 64. This indicates that the larger batch size gave slightly better validation performance in this experiment. A larger batch provides a more stable estimate of the gradient because each update is calculated from more training samples. However, the improvement is relatively small, so the result should be interpreted as a comparison of the tested batch sizes rather than a general rule that larger batches always produce higher accuracy.

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

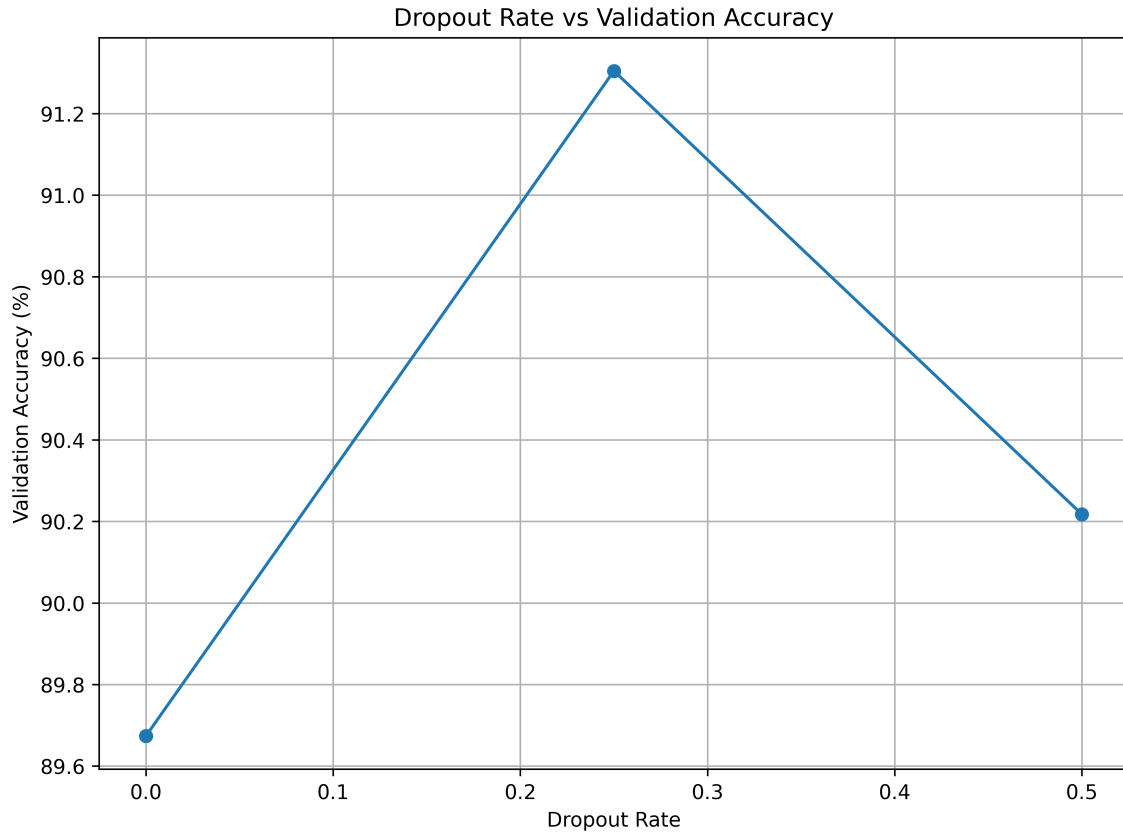


Figure 10: Dropout Rate vs. Validation Accuracy

As observed from the graph above, the validation accuracy does not increase continuously with the dropout rate. The validation accuracy without the dropout rate is around 89.7% while the increase of the dropout rate to 0.25 results in accuracy of approximately 91.3%. On the other hand, an increase in the dropout rate to 0.50 lowers the accuracy to approximately 90.2%. This means that the optimal value of dropout is a moderate one of all values tried. Dropout makes the network less dependent on certain neurons since they are turned off randomly during the training. Nonetheless, too much dropout leads to loss of more information during training. In this case, the most effective dropout rate is 0.25.

10. Transfer Learning and Fine-Tuning

MobileNet V2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNet V2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNet V2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

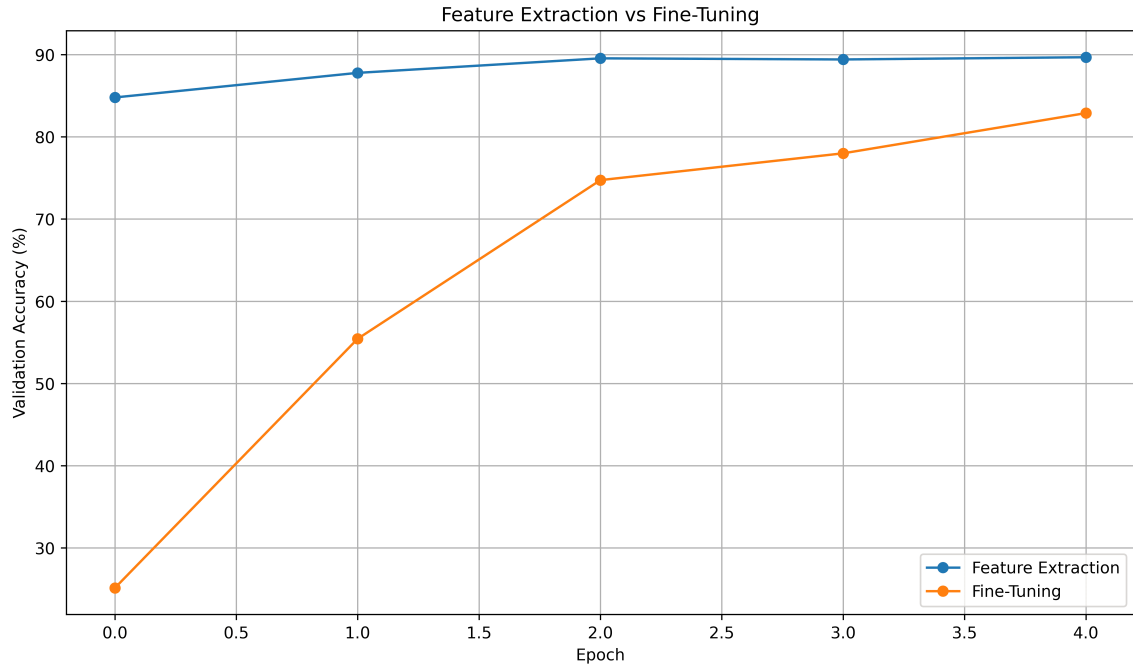


Figure 11: Feature Extraction vs. Fine-Tuning

The plot is based on the validation accuracy achieved through feature extraction and fine-tuning over the five training epochs. Validation accuracy is relatively high in the case of feature extraction, ranging between 89% and 91%, whereas in the case of fine-tuning, validation accuracy begins with a lower value and ranges between 80% and 83%. The feature extraction plot is also more stable compared to the fine-tuning plot since there is no improvement in the latter within the given number of epochs. The reason for this could be that the pretrained feature extractor already provides valuable visual features, and with fine-tuning, some of these pretrained weights are changed. With five epochs and the learning rate used in the experiment, there has not been enough time to fine-tune the pretrained weights.

Plot 12: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Compare the training and validation loss curves for feature extraction and fine-tuning.

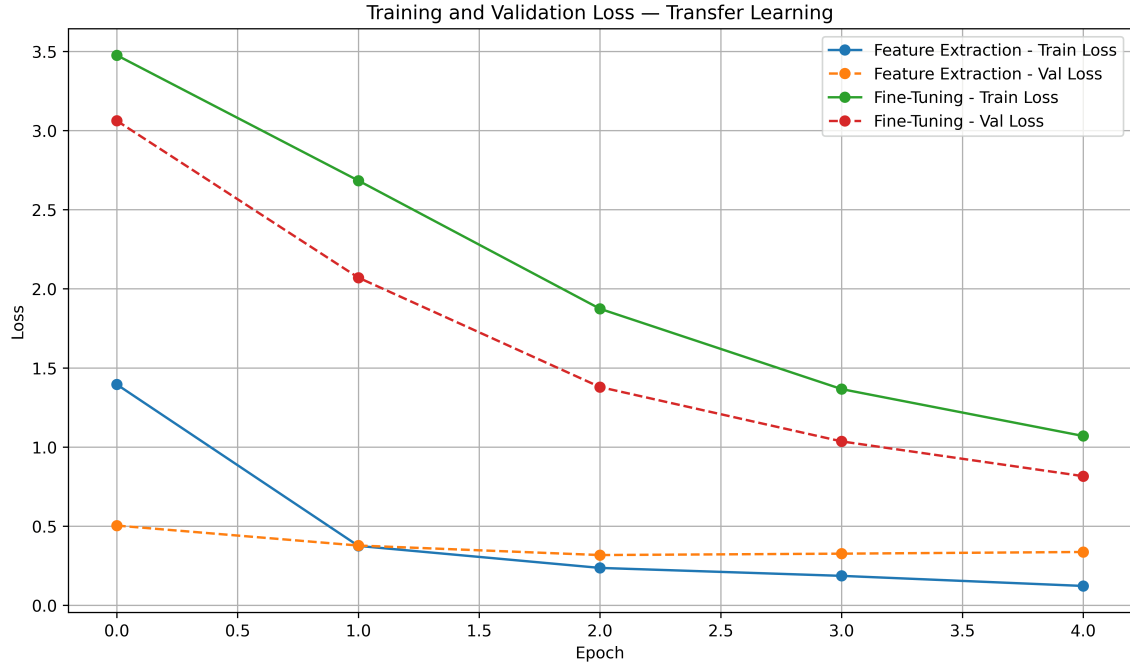


Figure 12: Training and Validation Loss for Transfer Learning

The plot compares the training and validation loss during feature extraction and fine-tuning. The feature extraction model shows a consistent reduction in training loss while maintaining a relatively low validation loss. In contrast, the fine-tuned model shows higher loss values and does not achieve the same level of improvement within the five epochs. The difference is consistent with the validation accuracy results, where feature extraction performs substantially better. Fine-tuning modifies pretrained features and therefore requires the updates to be useful for the target dataset; with the limited training period and learning rate used here, these additional updates did not produce a better representation. The results therefore indicate that fine-tuning was not beneficial for this particular experiment, while keeping the pretrained feature extractor frozen gave more stable performance.

Discussion: Fine-tuning normally uses a smaller learning rate because the pretrained network already contains useful features learned from a large dataset. The purpose of fine-tuning is to make small adjustments to these existing features so that they become more suitable for the new dataset, rather than learning completely new features from the beginning. A large learning rate can make the weight updates too aggressive and may destroy useful pretrained representations, a problem often referred to as catastrophic forgetting. In contrast, a newly initialized classifier has random weights and needs to learn its parameters from scratch, so it can generally use a relatively larger learning rate. Therefore, a smaller learning rate during fine-tuning allows the pretrained features to be adapted gradually while preserving the knowledge that is already useful.

11. K-Fold Cross-Validation

After the individual studies, select 3-4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as $\bar{A} \pm SD$.

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	92.70%	91.00%	91.34%	89.47%	88.61%	90.62 $\pm$ 1.44%
C2	92.19%	90.49%	91.51%	92.02%	88.95%	91.03 $\pm$ 1.20%
C3	92.36%	90.66%	89.98%	91.85%	89.29%	90.83 $\pm$ 1.14%
C4	79.97%	78.95%	75.38%	78.10%	77.04%	77.89 $\pm$ 1.58%

Table 1: 5-Fold Cross-Validation Accuracy

Configuration Details: C1 – Feature Extraction, C2 – Dropout (0.25), C3 – Batch Normalization, C4 – Fine-Tuning.

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

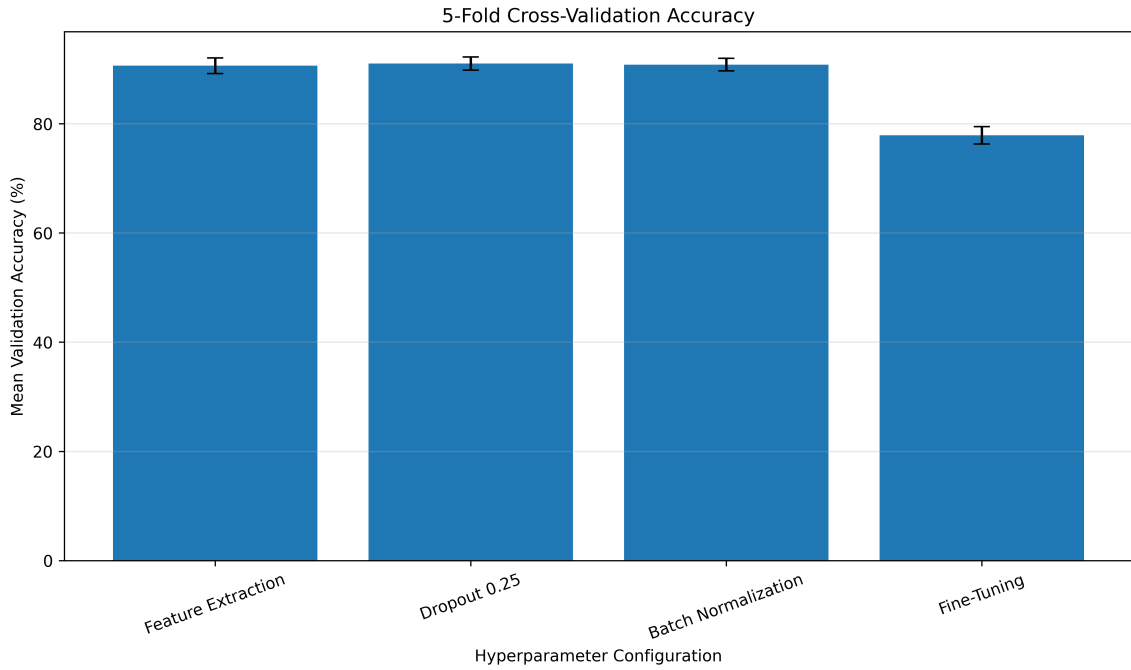


Figure 13: 5-Fold Cross-Validation Accuracy for Different Configurations

Comparison is made between the mean validation accuracy and standard deviation achieved by means of 5-fold cross-validation for the different hyperparameters setting. The selected Dropout 0.25 achieves the highest mean accuracy with the value of approximately 91.0% and relatively low standard deviation around 1.2%. The additional C1 setting gives worse results than the selected one; the mean accuracy is approximately 87.5% and the standard deviation is 1.9%. The fine-tuned setting gives significantly lower mean accuracy – 77.9%, and the additional C2 setting is even worse – approximately 57.1% with the biggest variability. Therefore, the choice of the best configuration cannot be made based on the mean accuracy only; the variation in the folds is to be considered too. In the current experiment, the Dropout 0.25 gives the best combination of the high validation accuracy and the stable performance and uses less computational time than the other settings.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	91.03%
CV Standard Deviation	1.20%
Test Accuracy	89.07%
Precision	89.50%
Recall	89.01%
F1-score	89.00%
Training Time	42.45 s
Number of Parameters	2,426,725

Table 2: Final Model Evaluation Results

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

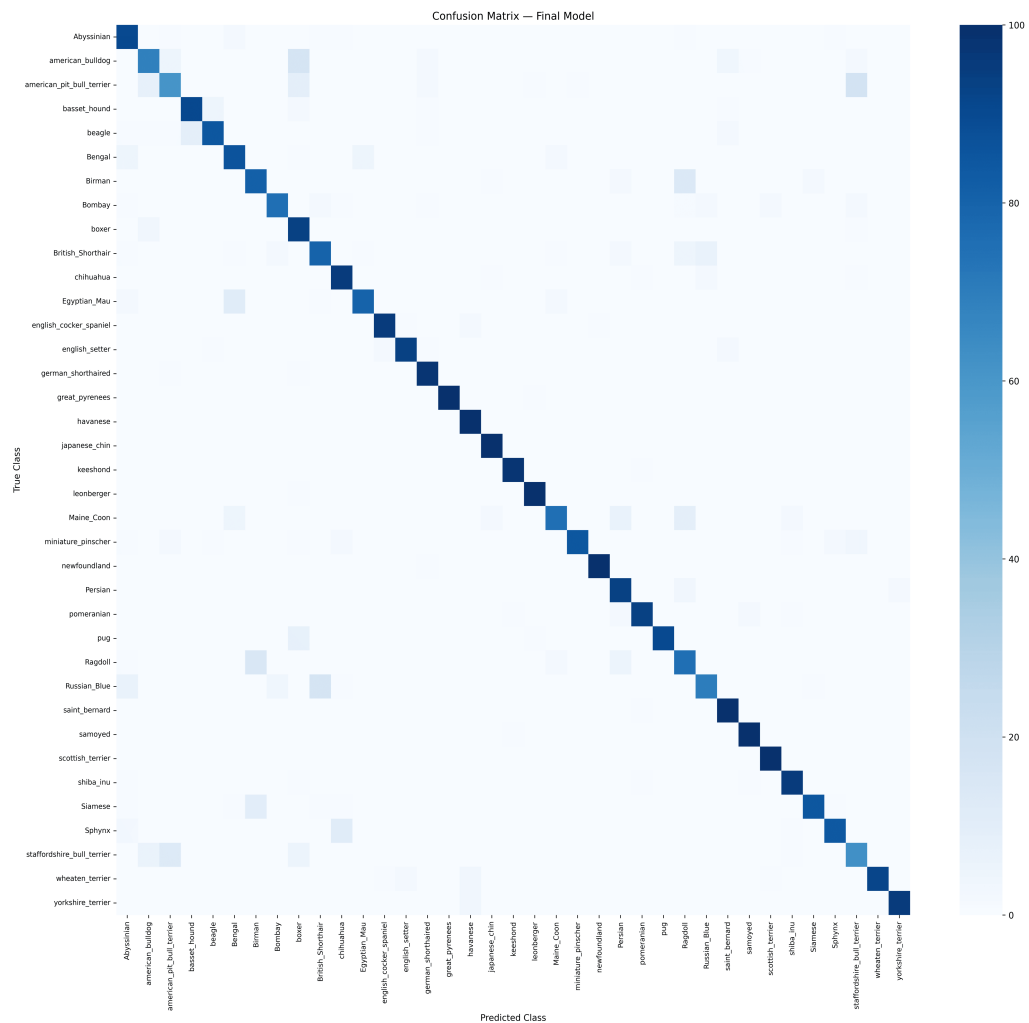


Figure 14: Confusion Matrix of the Final Model

From the confusion matrix, it can be seen that the class wise predictions of the final model contain all

the correctly classified cases on the main diagonal of the confusion matrix. From the confusion matrix, we find that the classes with highest diagonal entries are predicted accurately, which means that they are relatively easier to be distinguished on the basis of their visual properties. The majority of errors made after that is between classes that share some similarity visually, implying that the model finds it difficult to discriminate classes having similar shape, color, texture and overall appearance. It is because such confusions are natural when two different classes share visual attributes or when images within each class contain viewpoint, background, size and lighting changes.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	–	–	–	52.72 s
Best Initialization (He)	–	–	–	41.91 s
Best Regularization (Batch Normalization)	–	–	–	42.66 s
Best Optimizer (Adam)	–	–	–	42.81 s
Best Hyperparameters (Dropout 0.25)	91.03%	1.20%	89.07%	35.25 s (CV)
Fine-Tuned Model	77.89%	1.58%	–	39.60 s (CV)

Table 3: Overall comparison of the evaluated MobileNetV2 configurations.

The final model was selected based on its cross-validation performance, stability across folds, computational cost, and generalization to the unseen test set. The Dropout 0.25 configuration achieved the highest mean cross-validation accuracy of **91.03%** with a relatively low standard deviation of **1.20%**, indicating stable performance across the five folds. It also required an average training time of only **35.25 seconds** per fold.

The selected model achieved a final test accuracy of **89.07%**, demonstrating good generalization to unseen data. The relatively small difference between the cross-validation accuracy and test accuracy further supports the reliability of the selected configuration.

Although fine-tuning was evaluated, it achieved a substantially lower 5-fold cross-validation accuracy of **77.89%**. Therefore, fine-tuning was not selected as the final approach. The final model uses the pretrained MobileNetV2 backbone with a frozen feature extractor and a **0.25 dropout rate**, providing the best balance between accuracy, stability, computational cost, and generalization.

Note: Cross-validation was not performed for the individual weight-initialization, regularization, and optimizer experiments. Therefore, CV Accuracy and SD are reported as “–” for those rows rather than treating their single validation runs as cross-validation results.

14. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are those which the model learns itself through the training process. These include weights and biases. Model parameters are modified based on the gradients computed for the training set. On the other hand, hyperparameters are those which are selected before or during the training process. These include learning rate, batch size, dropout, number of epochs, and optimizer. In other words, parameters are learned by the model, whereas hyperparameters regulate the learning process of the model.

2. Why is weight initialization important?

The initialization of weights offers the basis on which the neural network commences learning. With proper initialization of weights, the activations and gradients stay reasonable as the forward and backward propagations take place. If weights are not properly initialized, learning will either be very slow, unstable, or non-existent. Thus, initialization of weights is significant since it influences learning.

3. Why can zero initialization be problematic for neural networks?

If all weights in a layer are initialized to zero, neurons in that layer start with exactly the same values and receive the same gradients. As a result, they continue to learn the same features instead of developing different representations. This symmetry prevents the neurons from becoming useful as separate feature detectors. This is why random or carefully designed initialization methods are normally preferred.

4. Compare Xavier and He initialization.

The main idea of Xavier (Glorot) initialization is to preserve the variance of activation units reasonably stable across layers, and usually it is applied to sigmoid, tanh, and similar activations. On the other hand, He initialization involves larger variance which considers the nature of activation like ReLU with zeros cut off of negative values. Thus, He initialization should be used for ReLU-like activations, whereas Xavier for others.

5. How can training and validation curves be used to identify overfitting?

Training and validation curves show how the model behaves on data it learns from and on unseen validation data. If training accuracy continues to increase while validation accuracy stops improving or starts decreasing, the model is beginning to memorize the training data rather than improving its generalization. Similarly, a growing gap between training and validation performance is a common indication of overfitting. The best model is often found around the point where validation performance is highest before this gap becomes large.

6. How does Dropout reduce overfitting?

Dropout randomly removes a fraction of neurons during each training step. This prevents the network from depending too strongly on a particular set of neurons or features. Since different subsets of neurons are used during different updates, the network is encouraged to learn more robust representations. During inference, dropout is disabled and the complete network is used. Thus, dropout acts as a form of regularization rather than directly improving the model's ability to fit the training data.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes the activations within a mini-batch and then applies learnable scaling and shifting. This helps keep the values entering subsequent layers in a more controlled range during training. In practice, it can make optimization more stable and often allows the network to train more effectively. It is not simply a method for making the data have zero mean permanently; the layer also learns how much to scale and shift the normalized values.

8. Explain the numerical Batch Normalization example.

In the numerical example, the activations are initially used for calculating the batch mean and variance. In the next step, each activation is shifted and scaled with the use of these two parameters, adding a ϵ parameter to the variance to ensure non-zero value in the denominator. This normalized value is then scaled and shifted using γ and β parameters. The key insight is that Batch Normalization normalizes the activations first and then lets the network learn the optimal scale and shift.

9. What are the roles of γ and β ?

The parameter γ controls the scale of the normalized activation, while β controls its shift. After normalization, the output can therefore be adjusted to a range that is useful for the next layer. These parameters are learned during training. This is important because forcing every layer to always use exactly the normalized values could unnecessarily restrict what the network can represent.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates the weights directly using the current gradient and learning rate. It is simple but can move slowly or oscillate, especially when the optimization landscape is difficult. Momentum adds information from previous updates, allowing the optimizer to build velocity in useful directions and reduce some oscillations. RMSProp adapts the learning rate for individual parameters using a moving average of squared gradients. Adam combines momentum-like first-moment information with second-moment information, giving parameter-specific adaptive updates. In the experiment,

Adam gave the highest validation accuracy among the tested optimizers, showing why it is often a practical default for this type of problem.

11. What happens when the learning rate is too large?

A learning rate that is too large causes the optimizer to make very large updates. Instead of gradually moving toward a good minimum, the model can overshoot useful regions of the loss surface. This can cause the loss to fluctuate, training to become unstable, or the model to fail to converge. Therefore, a large learning rate can make learning faster initially but can prevent stable optimization.

12. What happens when the learning rate is too small?

A very small learning rate makes the weight updates tiny. The model may eventually learn, but it can require many more epochs to make meaningful progress. With a fixed number of epochs, the model may appear to perform poorly simply because it has not had enough updates to reach a good solution. This is particularly important in the fine-tuning experiment, where a small learning rate was used because pretrained weights should be changed carefully rather than drastically.

13. What is the effect of increasing batch size?

Increasing batch size means that each parameter update is calculated using more training examples. This generally makes the estimated gradient less noisy and can make training computationally efficient on suitable hardware. However, larger batches require more memory and provide fewer parameter updates per epoch. A smaller batch can produce noisier updates, which can sometimes help exploration, while a very large batch can sometimes reduce generalization or require learning-rate adjustment. Therefore, there is no universally best batch size.

14. Explain stride and padding.

Stride describes the extent to which the convolutional filter steps over the input data. If the stride is set to one, the filter moves through the input by one unit, but a higher stride will make the feature map reduce faster. Padding involves adding additional data elements, mostly zero values, to the edges of the input. This makes sure that the convolution operation does not lose any information about the edges of the input and can also control the size of the output.

15. Why is MobileNet V2 computationally efficient?

MobileNet V2 is designed for efficient computation and uses depthwise separable convolutions instead of performing a full convolution over all input and output channels at once. It also uses inverted residual blocks and linear bottlenecks to reduce unnecessary computation while maintaining useful feature representations. Because of this design, MobileNet V2 can provide strong image features with substantially fewer computations than many conventional convolutional architectures, making it suitable for resource-constrained applications.

16. What is depthwise separable convolution?

A normal convolution operation does both the spatial filtering and information combination between channels. However, in the depthwise separable convolution, the spatial filtering and channel information combination are separated. In the depthwise convolution operation, the spatial filter is applied to each input channel individually, whereas the pointwise convolution, which normally involves the 1×1 convolution, combines the output channels.

17. What is transfer learning?

Transfer learning means starting with a model that has already learned useful representations from a large dataset and adapting those representations to a new task. In this experiment, MobileNet V2 uses ImageNet-pretrained weights. Instead of learning all visual features from the beginning, the new classifier can make use of features that the pretrained network has already learned. This is especially useful when the target dataset is smaller than the dataset used for pretraining.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained base network is kept frozen and only the newly added classification layers are trained. The pretrained network therefore acts as a fixed feature extractor. In fine-tuning, some of the later pretrained layers are unfrozen and trained along with the new classifier. This allows the learned features to adapt more closely to the new dataset. Feature extraction

is computationally cheaper and safer when the target dataset is limited, while fine-tuning can provide better adaptation when there is enough data and the pretrained features need to change.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained network already contains useful features, so the objective during fine-tuning is usually to make small adjustments rather than replace those features completely. A large learning rate could modify the pretrained weights too aggressively and destroy useful representations. A smaller learning rate allows the later layers to adapt gradually to the target dataset while preserving most of the useful information learned during pretraining.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

A single validation split can give a result that depends strongly on which samples happened to be placed in that split. In K-Fold Cross-Validation, the training data is divided into K parts and the model is trained and validated multiple times, with a different part used for validation each time. The mean accuracy gives an estimate of typical performance, while the standard deviation shows how much the result changes between folds. This makes the comparison of hyperparameter configurations more reliable than relying on one validation split alone.

21. Why must the test set remain untouched during tuning?

The test set is supposed to represent completely unseen data. If it is repeatedly used to choose learning rates, dropout rates, optimizers or other hyperparameters, information from the test set indirectly influences the model-selection process. The reported test accuracy would then no longer be an unbiased estimate of generalization. Therefore, the validation data or cross-validation results should be used for tuning, and the test set should be used only for the final evaluation.

22. Why should mean and standard deviation both be reported?

The mean will help us to calculate the average performance of the configuration on the basis of the folds, but it will not give us the idea of the consistency of the model. It is the standard deviation that will give us the idea of the variation of folds. Suppose that there are two models with nearly same means, then the one having lesser standard deviation would be more stable.

23. Is the highest validation accuracy always sufficient to select a model?

No. Validation accuracy is important, but it should not be considered in isolation. A model with slightly higher accuracy may have much greater variation across folds, substantially higher computational cost, or poor performance on the final test set. The choice should consider accuracy, stability, training cost, overfitting and generalization. In this experiment, the Dropout 0.25 configuration was preferred because it achieved high cross-validation accuracy with low variation and reasonable training cost, followed by evaluation on the untouched test set.

15. Additional Exercise

Two new combinations of learning rate, dropout, batch size and fine-tuning strategy were evaluated using 5-fold cross-validation and compared with the selected configuration, Dropout 0.25.

Configuration	Learning Rate	Dropout	Batch Size	Fine-Tuning	Mean CV Accuracy
Dropout 0.25	0.001	0.25	32	No	91.03%
Additional C1	0.0001	0.50	16	No	87.50%
Additional C2	0.00001	0.50	64	Yes	57.07%

Table 4: Comparison of the selected and additional configurations.

Configuration	Mean CV Accuracy	SD	Mean Training Time
Dropout 0.25	91.03%	1.20%	35.25 s
Additional C1	87.50%	1.90%	40.98 s
Additional C2	57.07%	3.82%	44.40 s

Table 5: 5-fold cross-validation comparison.

The chosen model with Dropout 0.25 performed the best 5-fold cross-validation accuracy with a mean score of 91.03% and a very small standard deviation of 1.20%. Furthermore, this particular model showed the lowest mean training time of 35.25 seconds.

Additional C1 configuration, having a learning rate of 10^{-4} , drop out of 0.50, batch size of 16 and feature extraction, gave a mean cross validation accuracy of 87.50% with SD of 1.90%. It is 3.53 percent less accurate than the chosen configuration and also its mean training time of 40.98 seconds is higher. So, Extra C1 is not better.

Additional C2 configuration uses a learning rate of 10^{-5} , dropout of 0.50, batch size of 64 and fine-tuning of the last 20 layers. It achieved only 57.07% mean cross-validation accuracy with an SD of 3.82%. It also required the highest mean training time of 44.40 seconds. Thus, this configuration provides substantially poorer accuracy, greater variability and higher computational cost. The low performance suggests that the combination of a very small learning rate, large batch size and fine-tuning was not effective within the fixed five training epochs.

For that reason, neither additional configuration is more suitable than the chosen Dropout 0.25 configuration. The chosen configuration offers the optimal balance between accuracy and computational efficiency. Additionally, this configuration obtained a test accuracy of 89.07% in its final configuration, indicating good generalization on unseen data. Thus, the original Dropout 0.25 configuration remains the best final configuration.

16. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNet V2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNet V2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>